

KUBEEDGEINFER

# A Closed-Loop, eBPF-Driven Kubernetes Framework for Heterogeneous Distributed LLM Inference

Turning the one-time model split into a continuous, self-correcting decision driven by kernel-level measurements.

eBPF / CO-RE

Kubernetes operator

Pipeline parallelism

Consumer edge hardware

**Research Presentation**

Author:   Institution:   Date:

## INTRODUCTION

# Edge LLM inference is a moving target

### 1 Why split at all

A 7B+ model does not fit on one consumer device. Pipeline parallelism spreads contiguous layer ranges across several machines, so the split point is the central performance decision.

### 2 Why the edge is different

Datacenter schedulers assume stable, homogeneous, well-cooled nodes. An edge cluster is laptops and hobby GPU boxes on WiFi: they thermally throttle, their links wander, and they disappear.

### 3 What existing frameworks do

They profile once, solve the placement problem once, and never revisit it. The plan is optimal for the machine state at  $t = 0$  and progressively wrong afterwards.

## THE CONSEQUENCE

**2.4×**

bottleneck-stage cost inflicted by a single throttled node under a frozen split

**4.3×**

when a degraded link and a throttled GPU coincide

**0**

requests served once a node dies and nothing re-plans

*Figures from the partitioner-level evaluation on slide 11 (12 layers, 3 stages).*

# Partitioning and placement

|                                                                          | CONTRIBUTION                                                                                                                                                                                               | LIMITATION RELEVANT HERE                                                                |
|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>EdgeShard</b><br><a href="#">IEEE IoT-J, 2024</a><br>arXiv:2405.14371 | Joint device selection and layer partition by dynamic programming across collaborative edge devices. Up to 50% latency reduction and $2\times$ throughput on Llama2 over 15 physical devices.              | <i>Profiles device speed and bandwidth offline; the DP runs on that static profile.</i> |
| <b>Galaxy</b><br><a href="#">IEEE INFOCOM, 2024</a><br>arXiv:2405.17245  | Hybrid tensor/sequence parallelism for in-situ Transformer inference, with heterogeneity-aware planning and tile-based compute/communication overlap. Up to $2.5\times$ lower latency.                     | <i>Planning is a one-shot offline step; no runtime re-planning under drift.</i>         |
| <b>Helix</b><br><a href="#">ACM ASPLOS, 2025</a><br>arXiv:2406.01566     | Models heterogeneous GPU serving as max-flow on a weighted graph and solves placement and request scheduling jointly by MILP. Up to $3.3\times$ throughput on 24–42 node clusters.                         | <i>Datacenter-scale GPUs; MILP is too heavy for a seconds-scale control loop.</i>       |
| <b>TPI-LLM</b><br><a href="#">arXiv, 2024</a><br>arXiv:2410.00531        | Argues tensor parallelism beats pipeline on low-resource devices; a sliding-window memory scheduler and star-based allreduce keep 70B-scale weights moving through small RAM.                              | <i>Optimises memory and collectives, not adaptation to node degradation.</i>            |
| <b>prima.cpp</b><br><a href="#">arXiv, 2025</a><br>arXiv:2504.08791      | 30–70B inference on real home clusters: the Halda scheduler co-optimises CPU/GPU workload and device selection; pipelined-ring parallelism hides disk I/O. $5\text{--}17\times$ lower TPOT than llama.cpp. | <i>Scheduling is decided at load time from static device capability.</i>                |

# Adaptation, decentralisation, and eBPF as control

|                                                                                                              | CONTRIBUTION                                                                                                                                                               | LIMITATION RELEVANT HERE                                                                      |
|--------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>LLM Partitioning at the Edge</b><br><a href="#">arXiv, 2025</a><br>arXiv:2505.02533                       | Partitions the decoder at attention-head granularity, co-locating each head with its K/V cache and migrating heads when memory tightens. Within 15–20% of an exact solver. | <i>Migration is triggered by memory pressure only – not thermal or network state.</i>         |
| <b>Parallax</b><br><a href="#">arXiv, 2025</a><br>arXiv:2509.26182                                           | Inference service over a decentralised pool of volunteer, non-uniform machines, addressing placement and routing without a central trusted cluster.                        | <i>Decentralised placement, but no kernel-level measurement substrate.</i>                    |
| <b>Adaptive Layer Splitting (MBRL)</b><br><a href="#">FITEE, Springer, 2025</a><br>doi:10.1631/FITEE.2400468 | Model-based reinforcement learning chooses the split point for wireless LLM inference as channel quality varies – explicitly an adaptive, not one-shot, formulation.       | <i>Learned policy needs training and gives no optimality guarantee; a single split point.</i> |
| <b>Distributed LLMs &amp; MLLMs Survey</b><br><a href="#">arXiv, 2025</a><br>arXiv:2503.16585                | Survey of distributed inference across advances, challenges and directions; names dynamic adaptation under heterogeneity as an open problem.                               | <i>Confirms the gap this work targets rather than filling it.</i>                             |
| <b>Agentic OS / sched_ext</b><br><a href="#">arXiv, 2025</a><br>arXiv:2509.01245                             | Custom Linux schedulers loaded at runtime as eBPF programs (sched_ext, Linux 6.12) – evidence that eBPF is becoming a control substrate, not only an observability one.    | <i>Schedules CPU tasks on one host; no notion of a distributed model pipeline.</i>            |

All ten works published 2024–2025; full citations on slide 15.

# Where the literature stops

| Capability                    | Datacenter serving (Helix) | Edge partitioning (EdgeShard, Galaxy, prima.cpp) | Adaptive splitting (MBRL, head-level) | KubeEdgeInfer                        |
|-------------------------------|----------------------------|--------------------------------------------------|---------------------------------------|--------------------------------------|
| Heterogeneity-aware placement | yes                        | yes                                              | yes                                   | yes                                  |
| Exact optimum for a snapshot  | MILP                       | DP                                               | no (learned / heuristic)              | DP, unit-tested vs brute force       |
| Re-plans while serving        | request routing only       | no                                               | yes                                   | yes, every 2 s                       |
| Kernel-measured network input | no                         | user-space probes                                | channel estimate                      | eBPF tp_btf / tcp_probe sRTT         |
| Thermal / throttle input      | no                         | no                                               | no                                    | NVML or /sys/class/thermal           |
| Survives node loss            | replication                | no                                               | no                                    | heartbeat-forced repartition         |
| Applies without restart       | n/a                        | no                                               | no                                    | gRPC hot reassign, generation-fenced |

*No prior system closes the loop from kernel-level measurement back to the layer split while inference is running.*

## PROBLEM STATEMENT

Given  $N$  transformer layers and  $K$  heterogeneous edge nodes whose compute speed, link latency and liveness all vary during inference, continuously maintain a contiguous layer assignment that minimises the bottleneck stage --- without restarting the pipeline.

### 1 Measure

Obtain per-flow latency and per-node thermal state cheaply and without instrumenting the inference code.

### 2 Decide

Solve the placement exactly at telemetry rate, and suppress oscillation when the measurement is merely noisy.

### 3 Act

Move layer ranges between live workers without dropping in-flight requests or restarting pods.

### 4 Heal

Detect node loss and redistribute its layers across the survivors automatically.

## CONSTRAINT

Commodity Linux only.

No datacenter interconnect, no vendor telemetry agent, no change to the model code, and no assumption that a node stays alive.

## CONTRIBUTIONS

# What is new here

### 1 Kernel telemetry as a scheduling input

CO-RE eBPF programs (tp\_btf/tcp\_probe, fentry/tcp\_sendmsg) feed per-flow smoothed RTT straight into a model-placement decision. Prior edge-LLM systems use user-space probes or offline profiles; none drives the partitioner from the kernel's own TCP state.

### 2 A closed loop, not a one-shot plan

WATCH-DECIDE-ACT-HEAL runs every 2s for the lifetime of the deployment. The split is a continuously maintained control variable rather than a deployment-time constant.

### 3 Hysteresis-gated exact optimisation

The  $O(N^2K)$  DP is exact for each snapshot; a 15% improvement threshold plus a 30s cooldown converts it into a stable controller. One gate handles thermal, network and failure triggers alike.

### 4 Restart-free, generation-fenced reassignment

Layer ranges move over gRPC while requests are in flight. Every assignment carries a generation; workers reject stale forwards and the stateless design makes context replay trivially correct.

### 5 An ablation that isolates continuous telemetry

A profile-once mode reuses the same DP and apply path but freezes its inputs, approximating offline-profiling systems – so the measured gain is attributable to live telemetry, not to a better solver.

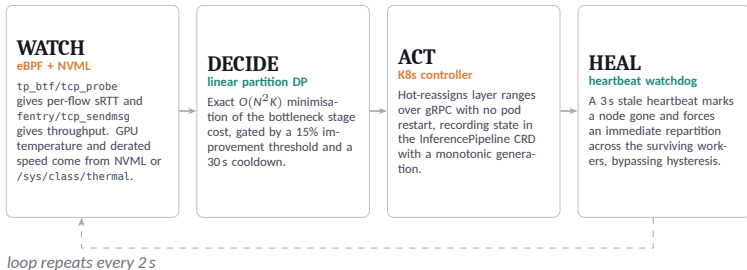
## CORE NOVELTY CLAIM

Not a better partitioning algorithm — the same classical DP the literature already uses, placed inside a kernel-measured feedback loop that never stops running.

*Items 1–3 are the primary novelty claims.*

## PROPOSED WORK

# The closed loop



**Consistency:** every assignment carries a generation. Workers reject stale-generation forwards; the router refetches the layout and replays the accumulated context. Workers are stateless, so replay is trivially correct.



# Formulation and stability

## Objective

$$\min_{i \in 1..K} \max \text{cost}(i)$$

$$\text{cost}(i) = \frac{\ell_i \cdot t_{\text{layer}}}{s_i} + \text{link}_{i-1}$$

subject to contiguous, non-overlapping ranges covering all  $N$  layers. An empty stage costs nothing — so the DP may bypass a worker entirely when its link has degraded past the value of its compute.

## Solution

$$f[j][w] = \min_{\text{split}} \max(f[\text{split}][w-1], \text{cost}(w, j - \text{split}))$$

The classical linear-partition recurrence: minimal bottleneck for the first  $j$  layers over the first  $w$  workers.  $O(N^2K)$  — about 1.7k operations for  $N=12$ ,  $K=3$ , so re-solving at 0.5 Hz is free. Verified against brute force on 200 randomised instances.

## Why a bare optimiser is not a controller

Telemetry is noisy. Re-applying the argmin on every 2 s tick would thrash the pipeline for sub-millisecond gains. Three rules turn the optimiser into a stable controller:

- **Improvement threshold**

Apply only if the new split beats the current one, re-evaluated under current telemetry, by 15%.

- **Cooldown**

At most one change per 30 s, so a transient spike cannot start an oscillation.

- **Membership override**

A changed worker set bypasses both gates and repartitions immediately — correctness beats stability.

Both thresholds are tunable live via `IMPROVEMENT_FRAC` and `COOLDOWN_S`.

## PROPOSED WORK

# Implementation

- **Node agent** Go + C (CO-RE BPF)

Privileged DaemonSet. Attaches `tp_btf/tcp_probe` and `fentry/tcp_sendmsg`, filtered to worker ports, and exports per-flow sRTT plus GPU state through a pluggable `gpu.Reader` (simulated / `cputherm` / `nvml`).

- **Controller** Go, controller-runtime

Aggregates telemetry, runs the Decider, writes assignments into the InferencePipeline CRD and pushes them to workers over gRPC. A watchdog forces repartition on a 3 s stale heartbeat.

- **Workers + router** Python

Stateless shard servers with pluggable `sim` and `gpt2` backends, on `WORKER_DEVICE=auto|cuda|cpu`. The router drives stages hub-and-spoke and replays context on a generation mismatch.

- **Benchmark harness** Python

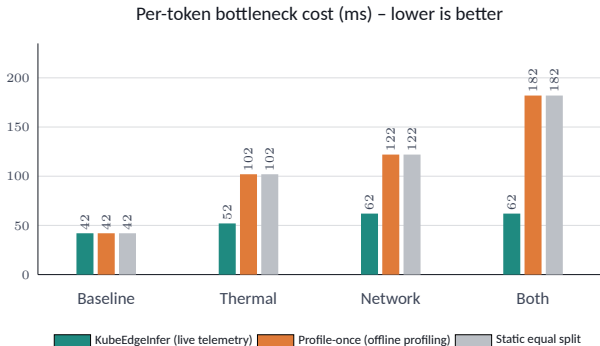
Five fault scenarios  $\times$  three modes, measuring bubble time, tokens/s and TTFT; plus a hysteresis sweep and a predicted-vs-measured fidelity check.

## Deployment reality

- eBPF is real, not simulated — CO-RE against the host BTF, loaded with cilium/ebpf.
- kind gives one host: real eBPF, loopback network, simulated GPU.
- k3s scripts turn laptops and GPU boxes on a LAN into the same cluster with zero YAML editing.
- `GPU_MODE=nvml` on NVIDIA nodes; `GPU_MODE=cputherm` reads `/sys/class/thermal` on GPU-less laptops.
- Distributed GPT-2 output is token-identical to single-process HuggingFace.

## RESULTS

# Bottleneck-stage cost under injected faults



**49%**

vs both baselines under thermal throttle

**49%**

vs both baselines under link degradation

**66%**

vs both baselines when the two coincide

Node failure: the static split is undefined over a worker set that no longer exists — the pipeline stops. KubeEdgeInfer heals to 62 ms.

Partitioner-level evaluation via `bench/dpsim` ( $N=12$  layers,  $K=3$  stages, 10 ms/layer,  $0.4\times$  throttle, 80 ms netem).

## RESULTS

# What the partitioner actually does

layer ranges, stage 1 / 2 / 3

---

|                 |     |     |      |              |
|-----------------|-----|-----|------|--------------|
| <b>Baseline</b> | 0-4 | 4-8 | 8-12 | <b>42 ms</b> |
|-----------------|-----|-----|------|--------------|

3 healthy stages

|                                    |     |     |      |              |
|------------------------------------|-----|-----|------|--------------|
| <b>Thermal throttle on stage 2</b> | 0-5 | 5-7 | 7-12 | <b>52 ms</b> |
|------------------------------------|-----|-----|------|--------------|

speed 1.0 -> 0.4

|                                 |     |       |      |              |
|---------------------------------|-----|-------|------|--------------|
| <b>80 ms delay into stage 2</b> | 0-6 | empty | 6-12 | <b>62 ms</b> |
|---------------------------------|-----|-------|------|--------------|

link 2 -> 82 ms

|                     |     |      |  |              |
|---------------------|-----|------|--|--------------|
| <b>Stage 3 lost</b> | 0-6 | 6-12 |  | <b>62 ms</b> |
|---------------------|-----|------|--|--------------|

3 workers -> 2

---

### Stage bypass

Under an 80 ms link the DP assigns stage 2 an empty range. Because an empty stage costs nothing — not even its hop — the router skips it entirely, and the two remaining nodes absorb all 12 layers. The optimiser discovers node exclusion without any special-case rule for it.

- **Optimality**

DP output matches brute force on 200 randomised instances.

- **Correctness**

3-stage distributed GPT-2 is token-identical to single-process greedy decoding.

- **Model fidelity**

Predicted vs measured bottleneck tracks at a stable 3.0–3.24× ratio: a

reliable ranking signal, not a calibrated latency.

*End-to-end tokens/s, TTFT and measured bubble time come from `bench/run.py`, which requires a live cluster.*

## COMPARISON

# Against the closest systems

|                       | EdgeShard<br>(IoT-J 24) | Galaxy<br>(INFOCOM 24) | Helix<br>(ASPLOS 25)  | prima.cpp<br>(2025)  | KubeEdgeInfer       |
|-----------------------|-------------------------|------------------------|-----------------------|----------------------|---------------------|
| Parallelism           | pipeline                | tensor +<br>sequence   | pipeline +<br>routing | pipelined ring       | pipeline            |
| Placement method      | DP (exact)              | heuristic<br>planner   | MILP on<br>max-flow   | Halda scheduler      | DP (exact)          |
| Telemetry source      | offline profile         | offline profile        | cluster profile       | device<br>capability | eBPF + NVML, live   |
| Re-plan while serving | no                      | no                     | routing only          | no                   | every 2 s           |
| Handles thermal drift | no                      | no                     | no                    | no                   | yes                 |
| Handles node loss     | no                      | no                     | replication           | no                   | yes, 3 s watchdog   |
| Target hardware       | 15 edge<br>devices      | edge devices           | 24–42 GPU<br>nodes    | home cluster         | consumer edge + k3s |
| Reported gain         | 50% latency             | 2.5× latency           | 3.3×<br>throughput    | 5–17× TPOT           | 49–66% bottleneck*  |

\* Bottleneck-stage cost vs a static split and a profile-once ablation; not directly comparable to the end-to-end figures in the other columns.

# The split point should be a control variable, not a deployment constant.

KubeEdgeInfer keeps a classical exact partitioner permanently in the loop, fed by kernel-measured latency and real thermal state, and applies its decisions to a running pipeline without a restart. On a heterogeneous edge cluster that is the difference between a plan that was right once and a plan that stays right.

### 1 Exactness is cheap

The DP costs microseconds; there is no reason to solve it only once.

### 2 Hysteresis is what makes it usable

One improvement threshold and one cooldown tame thermal, network and failure triggers alike.

### 3 Kernel telemetry needs no instrumentation

eBPF measures the real transport, not what the application thinks it sent.

### Future work

- A Llama-class backend so two 8 GB cards hold a real modern LLM across stages.
- Sustained multi-tenant load to observe genuine NVML throttling, not just flat curves.
- Non-contiguous and attention-head-level assignment, following [arXiv:2505.02533](https://arxiv.org/abs/2505.02533).
- Calibrating the cost model so the DP's bottleneck becomes a latency prediction, not only a ranking.

## REFERENCES

### All works cited

1. EdgeShard: Efficient LLM Inference via Collaborative Edge Computing. M. Zhang, J. Cao, X. Shen, Z. Cui. *IEEE Internet of Things Journal*, 2024. arXiv:2405.14371.
2. Galaxy: A Resource-Efficient Collaborative Edge AI System for In-situ Transformer Inference. *IEEE INFOCOM*, 2024. arXiv:2405.17245.
3. Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow. *ACM ASPLOS*, 2025. arXiv:2406.01566.
4. TPI-LLM: Serving 70B-scale LLMs Efficiently on Low-resource Edge Devices. 2024. arXiv:2410.00531.
5. Prima.cpp: Fast 30–70B LLM Inference on Heterogeneous and Low-Resource Home Clusters. 2025. arXiv:2504.08791.
6. Large Language Model Partitioning for Low-Latency Inference at the Edge. 2025. arXiv:2505.02533.
7. Parallax: Efficient LLM Inference Service over Decentralized Environment. 2025. arXiv:2509.26182.
8. Adaptive layer splitting for wireless large language model inference in edge computing: a model-based reinforcement learning approach. *Frontiers of Information Technology & Electronic Engineering*, Springer, 2025. doi:10.1631/FITEE.2400468.
9. Distributed LLMs and Multimodal Large Language Models: A Survey on Advances, Challenges, and Future Directions. 2025. arXiv:2503.16585.
10. Towards Agentic OS: An LLM Agent Framework for Linux Schedulers. 2025. arXiv:2509.01245.

Tooling: cilium/ebpf (CO-RE loader) · Kubernetes controller-runtime · NVIDIA NVML · k3s · Hugging Face Transformers (GPT-2).